

Comparing Item-to-Item and Profile-Based Content Recommendations

LLM4Rec - Assignment A02

Student: Kuznetsov Vladimir

Team: Individual

Email: vpkuznetsov_1@edu.hse.ru

Date: 2026-09-22

Assignment: A02 - Content-Based Movie Recommender

Abstract

This assignment extends a baseline content-based movie recommender using the MovieLens 100K dataset and compares Item-to-Item and Profile-Based recommendation. The implementation corrects the genre representation, replaces Jaccard similarity with normalized cosine similarity, and generates Top-5 recommendations from either one movie or an averaged profile of three watched movies. The experiments show a focused Item-to-Item result and a flatter similarity distribution when very different movies are averaged. Manual tests and rating-count analysis were also used to examine correctness, discovery, and data-quality limitations.

1. Introduction

The baseline recommender represented movies through genres. A02 extends it to compare Item-to-Item recommendation with a Profile-Based approach based on three watched movies.

Baseline inspection found an off-by-one error: `fields.slice(5, 24)` included the MovieLens unknown flag, shifted the genre mapping, and omitted Western. The implementation corrected this to `fields.slice(6, 24)`, replaced Jaccard with cosine similarity, added shared Top-5 ranking, excluded watched items, and calculated rating counts for analysis.

The final system was evaluated through read-only numerical reproduction and manual browser tests, focusing on correctness, score distributions, popularity/discovery, and coherent versus divergent profile averaging.

2. Related Work

MovieLens 100K contains 100,000 ratings and is a common recommender-system benchmark [1, 2].

Content-based recommenders compare item features with a representation of user preference; genre vectors provide a simple, interpretable representation [3]. Cosine similarity normalizes vector comparison for vector length [4].

3. Method

3.1 Feature representation

Each movie is represented by an 18-dimensional binary genre vector from Action through Western. The corrected parser uses `fields.slice(6, 24)`. *Unforgiven* (1992), for example, is correctly represented as Western. Two records have no active genre features after correction: ID 267 (unknown) and ID 1373 (*Good Morning* (1971)).

3.2 Cosine similarity and ranking

For vectors a and b , $\cos(a,b) = (a \cdot b) / (||a|| \cdot ||b||)$. If either norm is zero, similarity is defined as 0. A shared `getTopN(queryVector, excludeIds, 5)` function scores eligible movies and returns the five highest cosine scores.

3.3 Recommendation modes

Item-to-Item: the selected movie vector is the query and its ID is excluded. Profile-Based: three different watched vectors are averaged element by element: $p = (v_1 + v_2 + v_3) / 3$. All three watched IDs are excluded before ranking.

3.4 Popularity measure

`ratingCount(itemId)` is the number of records in `u.data` for that movie. It is displayed for analysis but never changes ranking. Across the 1,682 parsed movies, the median count calculated from `u.data` was 27.

3.5 Tools, configuration, and artifacts

Tool / component	Role
OpenCode Web / AI coding agent	Audit, planning, minimal code changes, read-only verification
HTML / CSS / vanilla JavaScript	Application and recommendation logic
MovieLens 100K	Genre metadata and rating interactions
Python 3	Read-only numerical reproduction
Chrome	Manual browser testing

Repository: github.com/Valdemaro2025/content-based-movie-recommender

Branch: main

Report: [kuznetsov_vladimir_a02_report.pdf](#)

4. Experiments

4.1 Item-to-Item: Toy Story (1995)

Rank	Recommendation	Cosine	Ratings
1	Aladdin and the King of Thieves (1996)	1.0000	26
2	Aladdin (1992)	0.8660	219
3	Goofy Movie, A (1995)	0.8660	20
4	Santa Clause, The (1994)	0.8165	82
5	Home Alone (1990)	0.8165	137

Toy Story was excluded. The Top-5 score range was 0.1835. Manual browser testing reproduced the same list and scores.

4.2 Divergent Profile-Based recommendation

The profile combined Toy Story (1995), GoldenEye (1995), and Unforgiven (1992), intentionally mixing very different genre groups.

Rank	Recommendation	Cosine	Ratings
1	Muppet Treasure Island (1996)	0.6761	84
2	Space Jam (1996)	0.6761	93
3	Hercules (1997)	0.6761	66
4	Pagemaster, The (1994)	0.6761	12
5	Maverick (1994)	0.6547	128

None of the watched IDs appeared in the result. The Top-5 score range was only 0.0215. The same list was reproduced manually.

4.3 Effect of profile averaging

A coherent comparison profile used Star Wars (1977), Return of the Jedi (1983), and The Empire Strikes Back (1980).

Measure	Divergent profile	Coherent profile
Watched-to-profile cosine	0.6547 / 0.6547 / 0.3780	0.9891 / 0.9891 / 0.9631
Highest Top-5 score	0.6761	0.8847
Top-5 score range	0.0215	0.1185

In these two runs, averaging divergent vectors produced a lower and flatter distribution. This is an observation from the tested profiles, not a universal claim.

4.4 Popularity and discovery

Item-to-Item rating counts were 26, 219, 20, 82, 137 (median 82); divergent Profile-Based counts were 84, 93, 66, 12, 128 (median 84). Both lists contained a below-catalog-median item: Goofy Movie, A had 20 ratings and Pagemaster, The had 12.

4.5 Manual functional testing

Verification	Observed result
Item-to-Item Top-5	PASS - exactly five results matching numerical reproduction
Profile-Based Top-5	PASS - expected five results
Watched exclusion	PASS - IDs 1, 2, 203 absent from profile Top-5
Duplicate validation	PASS - duplicate selections rejected
Genre mapping / zero vector	PASS - Western restored; zero vectors handled without NaN
Popularity display	PASS - rating counts shown

5. Discussion

5.1 Recommendation behavior and profile averaging

Item-to-Item for Toy Story stayed concentrated around Animation, Children's, and Comedy and produced relatively high similarities. The divergent profile mixed unrelated genre groups and its Top-5 occupied a narrow 0.0215 range. The two Top-5 lists had no movies in common.

The coherent-versus-divergent comparison illustrates a limitation of simple averaging: similar watched vectors preserve a specific preference direction, while very different vectors spread weight across unrelated genres. Weighted profiles, multiple preference clusters, or richer features could address this but were outside A02.

5.2 Popularity and discovery

Rating count does not influence ranking. The tested lists included both frequently rated and below-median items, but the small number of runs is not sufficient for a general conclusion about popularity bias.

5.3 Data quality and encoding observations

Unknown movie entry. During manual testing, an item titled unknown appeared in the dropdown and initially looked like a parsing problem. Dataset inspection showed that it is an actual MovieLens record (ID 267) with the unknown indicator set and no active genre fields. After the mapping correction it has an all-zero 18-dimensional genre vector. The cosine implementation returns 0 when either norm is zero, so the record is handled safely and was not removed.

Corrupted movie titles. Manual inspection also found titles in which non-ASCII letters were displayed as replacement characters. Examples included [replacement] K[replacement]dum klaka (Cold Fever), JLG/JLG - autoportrait de d[replacement]cembre, and C'est arriv[replacement] pr[replacement]s de chez vous. This is documented as a title-decoding/display limitation. Similarity uses separate numeric genre fields, so the display problem does not change vectors, cosine scores, or ranking. Fixing title decoding was outside the recommendation-logic scope of A02.

Other limitations. Equal scores retain dataset order because no tie-breaker was added. The recommender also uses only genres, not rating values or richer movie metadata.

6. AI Usage Disclosure

An AI coding agent was used to inspect the project, identify defects, compare options, and make the selected minimal changes. It corrected feature representation, replaced Jaccard with normalized cosine similarity, added shared Top-5 and Profile-Based recommendation, excluded watched items, and added rating counts.

A separate read-only verification reproduced the calculations against MovieLens using Python. ChatGPT (GPT-5.6 Sol) was used for workflow guidance, result interpretation, manual test planning, Git/GitHub guidance, and report drafting.

Personally verified. I manually tested the Toy Story Item-to-Item Top-5, the Toy Story + GoldenEye + Unforgiven profile, watched exclusion, duplicate validation, the unknown record, and visible title-encoding anomalies. AI output was not treated as verification by itself.

Audit trail. The exported OpenCode session JSON submitted with the assignment records the coding-agent interaction and tool/file activity. ChatGPT activity outside OpenCode is disclosed separately above.

References

- [1] GroupLens Research, "MovieLens 100K Dataset." [Online]. Available: <https://grouplens.org/datasets/movielens/100k/>. [Accessed: 2026-09-22].
- [2] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," ACM Transactions on Interactive Intelligent Systems, vol. 5, no. 4, 2015. [Online]. Available: <https://doi.org/10.1145/2827872>. [Accessed: 2026-09-22].
- [3] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 3rd ed. Springer, 2022. [Online]. Available: <https://doi.org/10.1007/978-1-0716-2197-4>. [Accessed: 2026-09-22].
- [4] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008, ch. 6. [Online]. Available: <https://nlp.stanford.edu/IR-book/html/htmledition/dot-products-1.html>. [Accessed: 2026-09-22].
- [5] V. Kuznetsov, "content-based-movie-recommender," GitHub repository, 2026. [Online]. Available: <https://github.com/Valdemaro2025/content-based-movie-recommender>. [Accessed: 2026-09-22].